CBOW setup: from running text to training samples

A window slides over the text one token at a time; every position turns into one (context → target) example for the same network.

1. A corpus and its vocabulary

Toy corpus as a token stream

Tokenized and lower-cased; articles dropped to keep the vocabulary small

cat	sat	on	mat	dog	sat	in	mat
$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$	$t=7$	$t=8$

Vocabulary: $V = 6$ distinct words

cat	dog	sat	on	in	mat
row 1	row 2	row 3	row 4	row 5	row 6

Each word owns one trainable row $\boldsymbol{e}_w$ of W_{in}, and one row of W_{out}.

Repeated tokens (sat, mat) reuse the same row.

2. The window slides one token at a time

radius $m = 1$: one word on each side, so $C = 2m = 2$ context words

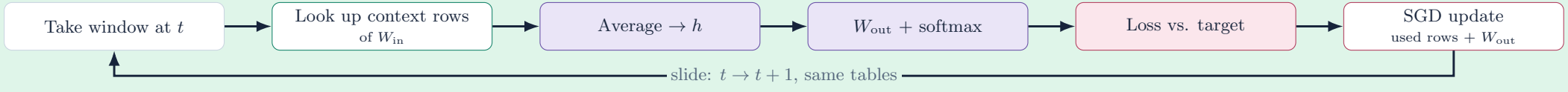
context target (center)

window of width $2m + 1 = 3$

$t=2$	**cat**	**sat**	**on**	mat	dog	sat	in	mat	context (**cat**, **on**) → target **sat**	**sample A → next slide, part 1**
$t=3$	cat	**sat**	**on**	**mat**	dog	sat	in	mat	context (sat, mat) → target on	also a training sample
$t=4$	cat	sat	**on**	**mat**	**dog**	sat	in	mat	context (on, dog) → target mat	also a training sample
$t=5$	cat	sat	on	**mat**	**dog**	**sat**	in	mat	context (mat, sat) → target dog	also a training sample
$t=6$	cat	sat	on	mat	**dog**	**sat**	**in**	mat	context (**dog**, **in**) → target **sat**	**sample B → next slide, part 3**
$t=7$	cat	sat	on	mat	dog	**sat**	**in**	**mat**	context (sat, mat) → target in	also a training sample

Edge positions $t = 1$ and $t = 8$ have a neighbour on one side only; implementations shorten the window there. One pass over all positions is an epoch.

3. What happens at every window position



The next slide follows samples A and B through one pass of this loop; W_{in} and W_{out} persist across all windows.

CBOW training: the connections change, but the weights persist

Each context window chooses rows from the SAME trainable embedding table. Backpropagation updates the rows used by that window.

1. Forward pass: sample A

context: **cat** … **on** target: **sat**

W_{in}: permanent embedding table
Word index ⟶ row of trainable weights

cat	e_{cat}	selected
dog	e_{dog}	trainable vector
sat	e_{sat}	trainable vector
on	e_{on}	selected
in	e_{in}	trainable vector
mat	e_{mat}	trainable vector

Average $h = \dfrac{e_{\text{cat}} + e_{\text{on}}}{2}$

Fixed coefficients, no trainable weights

$h \in \mathbb{R}^d$: h_1 h_2 $\cdots$ h_d

W_{out} dense weights

Same matrix every sample

Softmax

$P(\text{sat} \mid h)$ loss vs. target

2. Backpropagation / SGD on sample A

$$\frac{\partial L}{\partial e_{\text{cat}}} = \frac{\partial L}{\partial e_{\text{on}}} = \frac{1}{2}\frac{\partial L}{\partial h}$$

Inactive input rows (dog, sat, in, mat) receive no gradient in this example.

$e_{\text{cat}}, e_{\text{on}}$ ← h ← W_{out} ← L

Parameters updated by SGD:
Selected input vectors: e_{cat} and e_{on}
Shared output matrix: W_{out}

3. Next training sample: sample B

context: **dog** … **in** target: **sat**

W_{in}: the same table, now updated
Different active rows; persistent parameters

cat	e_{cat}	trainable vector
dog	e_{dog}	selected
sat	e_{sat}	trainable vector
on	e_{on}	trainable vector
in	e_{in}	selected
mat	e_{mat}	trainable vector

Average $h = \dfrac{e_{\text{dog}} + e_{\text{in}}}{2}$

Again, a fixed average

$h \in \mathbb{R}^d$: h_1 h_2 $\cdots$ h_d

W_{out} same layer

Learned from sample A and earlier samples

Softmax

$P(\text{sat} \mid h)$ new loss

Key idea

The apparent "rewiring" is sparse row selection from a persistent table, not a change in the model architecture.

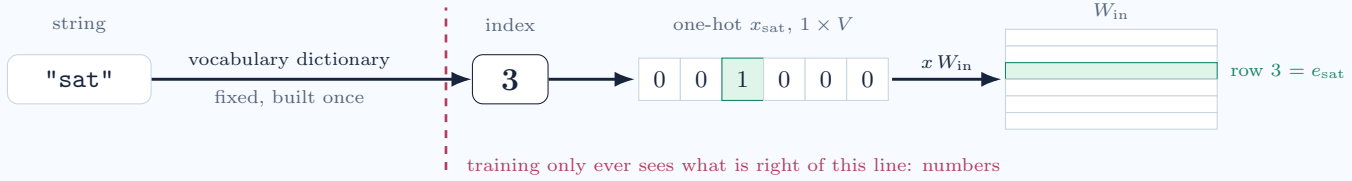
Next slide: the MLP interpretation, point by point.

Only the active input embeddings change per sample; both parameter tables remain stored and trainable.

The MLP interpretation, point by point

CBOW is a small neural network. Comparing it with a standard MLP and with CNN pooling shows what is the same, what only looks different, and what is borrowed.

1. The string never trains: a word *is* its index



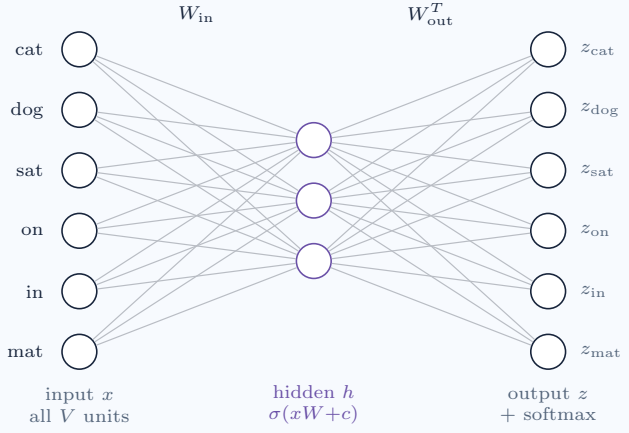
The string is a lookup key, used once, before training.

No gradient reaches the spelling; the dictionary is not trained.

Rename every “sat” to “w3”: the run is identical, number for number.

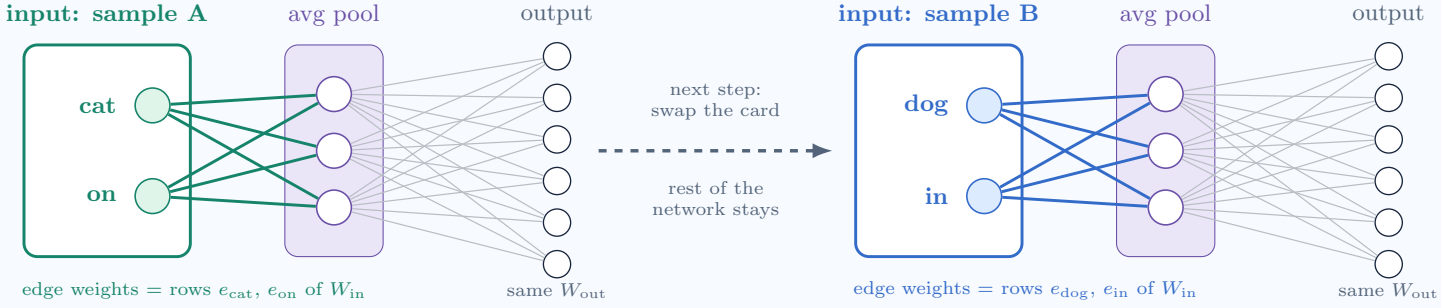
Spelling carries no meaning: “cat” and “cart” get unrelated rows.

2. A standard MLP



Fixed wiring: every input unit exists on every step; only input *values* change.

3. CBOW: as if the input layer were swapped each step



It looks as if each step plugs in a new input layer with only C units, wired to the same pooling and output layers.

Why only “almost”: the plugged-in weights are not new. They are rows of the persistent W_{in}, written back after SGD.

Exact view: the full MLP of panel 2 with input $b = s/C$. Inactive inputs are 0, so their wires carry nothing and receive no gradient. Hiding those idle units is what makes it look like the input layer was replaced.

4. The averaging stage is average pooling, the same tool you met in CNNs

not an MLP layer: no weights, no activation

Recall: 2×2 pooling in a CNN

1	3
2	6

max pool → **6**

avg pool → **3**

one window

GAP: avg pool over the whole map, per channel

CBOW: pool over the window’s words, coordinate by coordinate

e_{cat}	0.218	−1.572	···	0.294	a window of $C = 2$ “positions”
e_{on}	−0.418	0.732	···	1.106	
avg pool $= h$	−0.100	−0.420	···	0.700	
max pool	0.218	0.732	···	1.106	not used by CBOW

Carried over from CNN pooling

no trainable parameters, no activation

order-free: positions are forgotten (bag of words)

avg: every input gets $\frac{1}{C}\,\partial L/\partial h$

(max would feed only the winner, per coordinate)

Twist: the pooled inputs are parameters

CBOW mirrors a CNN head: features → global average pool → linear layer → softmax, except its “features” are rows of W_{in} themselves.

CBOW: each word carries a trainable d-dimensional embedding

Only the currently selected vectors are expanded below. Entries line up by coordinate, so the average can be read column by column.

1. Sample A: cat and on are selected

context: **cat** ⋯ **on** target: **sat**

W_{in}: persistent embedding table

cat	$\boldsymbol{e}_{\text{cat}}$	selected
dog	$\boldsymbol{e}_{\text{dog}}$	
sat	$\boldsymbol{e}_{\text{sat}}$	
on	$\boldsymbol{e}_{\text{on}}$	selected
in	$\boldsymbol{e}_{\text{in}}$	
mat	$\boldsymbol{e}_{\text{mat}}$	

Selected embeddings, and their component-wise average

d coordinates in every row

vector	1	2	⋯	d
$\boldsymbol{e}_{\text{cat}}$	0.218	−1.572	⋯	0.294
$\boldsymbol{e}_{\text{on}}$	−0.418	0.732	⋯	1.106
$\boldsymbol{h}_A$ **(avg)**	−0.100	−0.420	⋯	0.700

W_{out} + softmax

scores for all words

$\boldsymbol{h}_A \in \mathbb{R}^d$: the average preserves every coordinate (it is not a scalar).

2. Backpropagation on sample A

$\nabla_{\boldsymbol{h}_A} L = [\,0.200,\ -0.100,\ \ldots,\ 0.500\,] \in \mathbb{R}^d$

$\nabla_{\boldsymbol{e}_{\text{cat}}} L = \nabla_{\boldsymbol{e}_{\text{on}}} L = \frac{1}{2} \nabla_{\boldsymbol{h}_A} L$

SGD updates the two selected embedding rows using these gradients.

W_{out} is also trained; unused input rows do not change on this step.

3. Sample B: different rows, same network

context: **dog** ⋯ **in** target: **sat**

W_{in}: persistent embedding table

cat	$\boldsymbol{e}_{\text{cat}}$	
dog	$\boldsymbol{e}_{\text{dog}}$	selected
sat	$\boldsymbol{e}_{\text{sat}}$	
on	$\boldsymbol{e}_{\text{on}}$	
in	$\boldsymbol{e}_{\text{in}}$	selected
mat	$\boldsymbol{e}_{\text{mat}}$	

Selected embeddings, and their component-wise average

d coordinates in every row

vector	1	2	⋯	d
$\boldsymbol{e}_{\text{dog}}$	0.604	−0.280	⋯	0.016
$\boldsymbol{e}_{\text{in}}$	−0.344	0.900	⋯	−0.336
$\boldsymbol{h}_B$ **(avg)**	0.130	0.310	⋯	−0.160

W_{out} + softmax

scores for all words

$\boldsymbol{h}_B \in \mathbb{R}^d$: the average preserves every coordinate (it is not a scalar).

Key idea Each $\boldsymbol{e}_w$ is one fixed-size, trainable d-vector stored in W_{in}.

$\boldsymbol{h} = (\boldsymbol{e}_{w_1} + \cdots + \boldsymbol{e}_{w_C})/C \in \mathbb{R}^d$: average corresponding components, not the entire vector to one value.

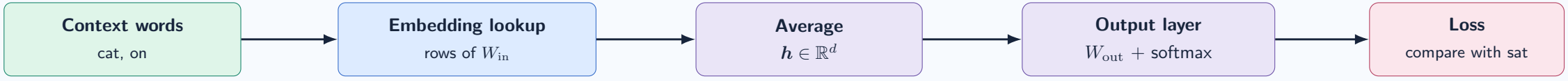
New context window = new selected rows; the stored weights and the architecture persist between training examples.

CBOW: from prediction task to word embeddings

Continuous Bag of Words learns one reusable vector per word by predicting a missing word from its context.

1. A prediction task creates the training signal

Example: context = **cat** and **on**; the correct target is **sat**.



The averaging operation has no trainable parameters; the input embeddings and output weights are trainable.

Backpropagation travels from the loss through W_{out} and $\boldsymbol{h}$ to the selected rows of W_{in}.

2. What the training actually changes

Input embedding matrix W_{in}

One learned d-vector per vocabulary word.

Only selected input rows receive a gradient in a step.

Prediction layer W_{out}

Scores candidate target words given $\boldsymbol{h}$.

Its trainable weights receive gradients as well.

3. After training: keep the product, not the predictor

KEEP W_{in}
persistent word embeddings

USUALLY DISCARD W_{out} + softmax
the training-time prediction head

The learned vectors can be reused in downstream models; normally no averaging or target prediction is needed to look up a word vector.

Main idea We train CBOW as a word predictor because prediction errors teach us useful word representations.

The lasting artifact is the embedding table; the output classifier is primarily a means of learning it.

CBOW as matrix multiplication: seven context words

The binary selection vector chooses seven distinct context words; the factor 1/7 performs the averaging.

1. Select seven words, then normalize for averaging

Example: "small cat quietly [sat] on the soft mat"; the target is sat, $C = 7$ and $V = 9$.

small	cat	quietly	sat	on	the	soft	mat	dog
1	1	1	0	1	1	1	1	0

$\boldsymbol{s} = [1, 1, 1, 0, 1, 1, 1, 1, 0]$: **binary selection vector,** $1 \times V$.

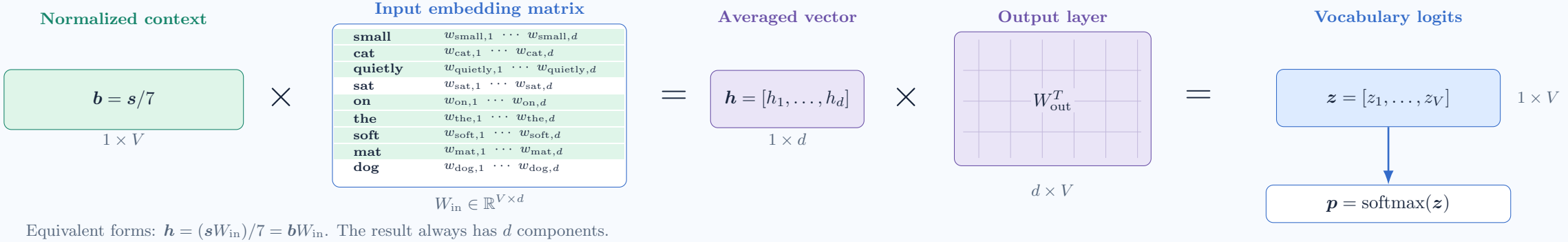
Averaging is a separate scale factor

$$\boldsymbol{b} = \frac{\boldsymbol{s}}{C} = \frac{1}{7}[1, 1, 1, 0, 1, 1, 1, 1, 0]$$

$$\boldsymbol{h} = \frac{\boldsymbol{s} W_{\text{in}}}{7} = \boldsymbol{b} W_{\text{in}}$$

Seven ones $\Rightarrow \sum_i s_i = 7$. Repeated tokens would require counts.

2. Forward pass: matrix multiplication and dimensions



Equivalent forms: $\boldsymbol{h} = (\boldsymbol{s} W_{\text{in}})/7 = \boldsymbol{b} W_{\text{in}}$. The result always has d components.

3. Why input-embedding gradients are sparse

$\boldsymbol{g} = \frac{\partial L}{\partial \boldsymbol{h}} \in \mathbb{R}^{1 \times d}$

$$\frac{\partial L}{\partial W_{\text{in}}} = \underbrace{\boldsymbol{b}^T}_{V \times 1} \underbrace{\boldsymbol{g}}_{1 \times d}$$

Outer product: the matrix gradient has shape $V \times d$.

The nine input-embedding gradient rows

Selected 7 words: $\boldsymbol{g}/7$ per row

Unselected 2 words (sat, dog): **0** per row

Only the seven selected words receive input-embedding gradients.

W_{out} also trains on this step.

One-line summary

$$\boldsymbol{p} = \text{softmax}\left(\tfrac{1}{7} \boldsymbol{s} W_{\text{in}} W_{\text{out}}^T\right)$$

Everything before softmax is linear; $W_{\text{in}} W_{\text{out}}^T$ has rank at most d.

Standard full-softmax CBOW shown. After training, typically retain W_{in} and discard the output prediction layer.

CBOW and multinomial logistic regression

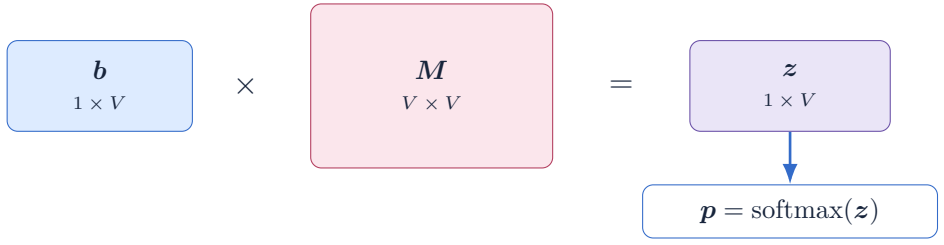
Both perform softmax classification. CBOW learns a factorized weight matrix and retains its intermediate embeddings.

1. The same classification problem, with two parameterizations

In both diagrams, $\boldsymbol{b} \in \mathbb{R}^{1\times V}$ is the normalized bag-of-words context input.

Multinomial logistic regression

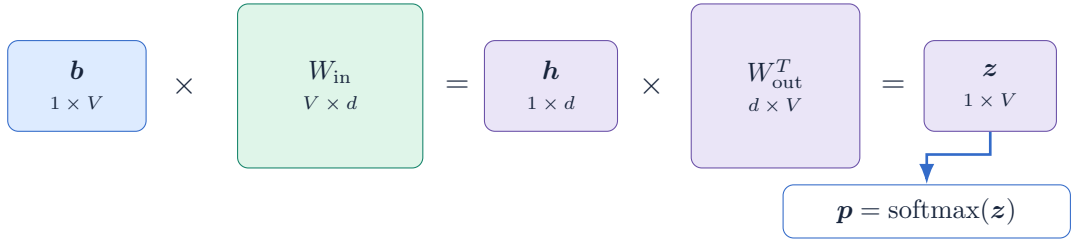
One $V \times V$ output-weight matrix



$\boldsymbol{p} = \text{softmax}(\boldsymbol{b}M + \boldsymbol{c})$ (bias $\boldsymbol{c}$ optional)

CBOW with full softmax

Two matrices joined through a d-dimensional bottleneck



$\boldsymbol{p} = \text{softmax}(\boldsymbol{b}W_{\text{in}}W_{\text{out}}^T)$ (no bias in standard CBOW)

2. The key relationship: a low-rank factorization

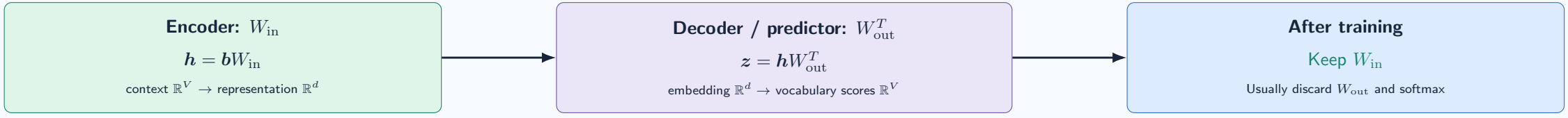
Softmax regression learns the full $V \times V$ matrix M; CBOW represents it as two smaller trainable matrices:

$$M = W_{\text{in}}W_{\text{out}}^T$$

$\text{rank}(M) \le d$; with $d \ll V$, CBOW has far fewer parameters and forces information through a d-dimensional representation.

3. Encoder / decoder interpretation

CBOW predicts the target word from its context; unlike an autoencoder, it does not attempt to reconstruct the input.



Both matrices learn through backpropagation, but the trained W_{in} supplies the word vectors we normally want to reuse.

The prediction network is a training scaffold: the learned embedding dictionary is the lasting product.

Skip-gram: the center word predicts its context

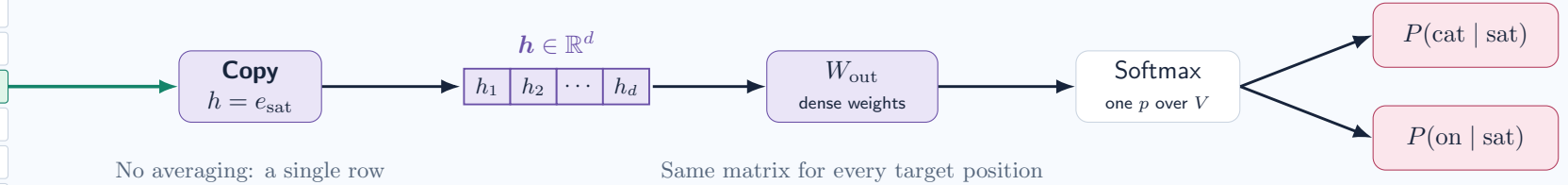
Same two tables as CBOW, opposite direction: ONE input row is looked up, and it must predict EACH surrounding word.

1. Forward pass: sample A

center: **sat** targets: **cat** … **on** (two training pairs)

W_{in}: permanent embedding table
Only the center word's row is read

cat	e_{cat}	trainable vector
dog	e_{dog}	trainable vector
sat	e_{sat}	selected
on	e_{on}	trainable vector
in	e_{in}	trainable vector
mat	e_{mat}	trainable vector



No averaging: a single row

Same matrix for every target position

2. Backpropagation / SGD on sample A

$$\frac{\partial L}{\partial e_{\text{sat}}} = \frac{\partial L_{\text{cat}}}{\partial h} + \frac{\partial L_{\text{on}}}{\partial h}$$

All other input rows (cat, dog, on, in, mat) receive no gradient: here the context words are targets, not inputs.



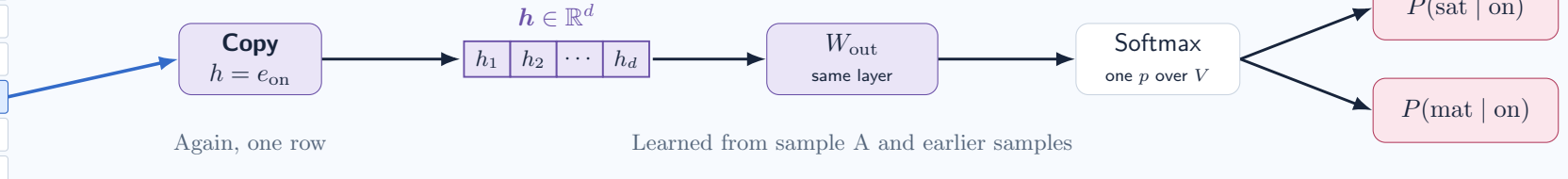
Parameters updated by SGD:
Center input vector: e_{sat}, with the gradient signals of both targets added
Shared output matrix: W_{out}

3. Slide the window: sample B

center: **on** targets: **sat** … **mat** (two more training pairs)

W_{in}: the same table, now updated
A different single row; persistent parameters

cat	e_{cat}	trainable vector
dog	e_{dog}	trainable vector
sat	e_{sat}	trainable vector
on	e_{on}	selected
in	e_{in}	trainable vector
mat	e_{mat}	trainable vector



Again, one row

Learned from sample A and earlier samples

CBOW versus Skip-gram

CBOW: several context rows → one average h → one prediction (the center word).

Skip-gram: one center row → $h = e_{\text{center}}$ → one prediction per context word, all from the same p.

Both store and train the same two tables, W_{in} and W_{out}; only the direction of prediction changes.

Skip-gram as matrix multiplication: one center word, seven targets

A one-hot vector selects a single row of W_{in}; the same output distribution is scored against every context word.

1. One-hot input, and seven training pairs

Same sentence as the CBOW matrix slide: "small cat quietly [sat] on the soft mat"; the center is sat, $C = 7$ and $V = 9$.

small	cat	quietly	sat	on	the	soft	mat	dog
0	0	0	1	0	0	0	0	0

$\boldsymbol{x} = [0,0,0,1,0,0,0,0,0]$: **one-hot center vector,** $1 \times V$.

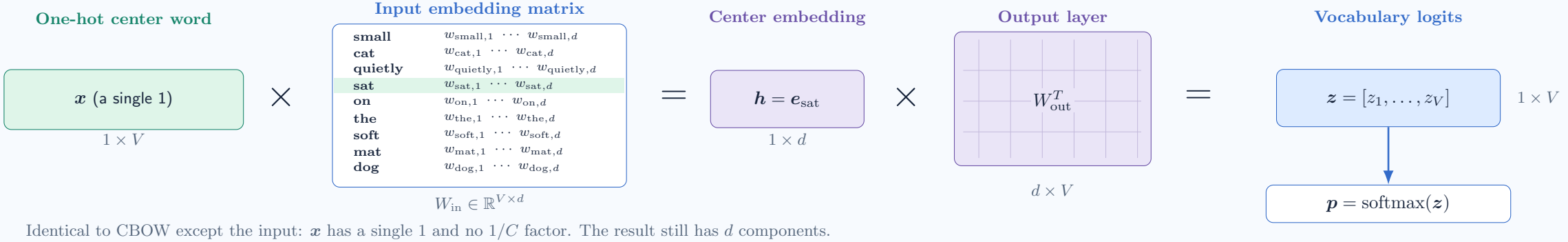
No averaging, no scale factor

$\boldsymbol{h} = \boldsymbol{x}W_{\text{in}} = \boldsymbol{e}_{\text{sat}}$

Pairs: (sat, small), (sat, cat), …, (sat, mat)

Seven targets share one $\boldsymbol{h}$ and therefore one $\boldsymbol{p}$.

2. Forward pass: matrix multiplication and dimensions



Identical to CBOW except the input: $\boldsymbol{x}$ has a single 1 and no $1/C$ factor. The result still has d components.

3. Seven losses, one input row

$$L = -\sum_{j=1}^{7} \log p_{c_j}, \qquad \partial L/\partial \boldsymbol{z} = 7\boldsymbol{p} - \sum_{j=1}^{7} \boldsymbol{y}_{c_j}$$

$$\boldsymbol{g} = \frac{\partial L}{\partial \boldsymbol{z}} W_{\text{out}} \in \mathbb{R}^{1 \times d}, \qquad \frac{\partial L}{\partial W_{\text{in}}} = \underbrace{\boldsymbol{x}^T}_{V \times 1} \underbrace{\boldsymbol{g}}_{1 \times d}$$

$\boldsymbol{y}_{c_j}$ is the one-hot vector of the j-th context word.

The nine input-embedding gradient rows

Center word (sat): $\boldsymbol{g}$, the sum of seven signals

Other 8 words: $\mathbf{0}$ per row

Compare CBOW (matrix slide): seven rows, each $\boldsymbol{g}/7$.

W_{out} also trains on this step.

One-line summary

$\boldsymbol{p} = \text{softmax}\left(\boldsymbol{x}W_{\text{in}}W_{\text{out}}^T\right)$, **scored against each of the C context words**

Same factorization $M = W_{\text{in}}W_{\text{out}}^T$ as CBOW: with a one-hot input, $\boldsymbol{z}$ is simply the row of M belonging to the center word.

Standard full-softmax Skip-gram shown. word2vec in practice uses negative sampling or hierarchical softmax, and samples the window size.

CBOW and Skip-gram: same tables, opposite direction

Both models are trained with one sliding context window; they differ in which side of the window is the input.

1. One window, two training recipes

Aspect	CBOW	Skip-gram
Prediction	context words → center word	center word → each context word
Rows of W_{in} read	C rows, averaged	1 row, copied
Hidden vector $\boldsymbol{h}$	$\frac{1}{C}(\boldsymbol{e}_{c_1} + \cdots + \boldsymbol{e}_{c_C})$	$\boldsymbol{e}_{\text{center}}$
Loss terms per window	1 (the center word)	C (one per context word)
W_{in} gradient	C rows, each $\boldsymbol{g}/C$	1 row, the sum of C signals
Cost per window	lower: one prediction	higher: C predictions
Usual strength	fast; smooths over the context	better vectors for rare words
Kept after training	W_{in}	W_{in}

2. One low-rank model, two kinds of input

CBOW

$\boldsymbol{p} = \text{softmax}(\boldsymbol{b} W_{\text{in}} W_{\text{out}}^T)$, $\boldsymbol{b} = \boldsymbol{s}/C$ (averaged bag of words)

Skip-gram

$\boldsymbol{p} = \text{softmax}(\boldsymbol{x} W_{\text{in}} W_{\text{out}}^T)$, $\boldsymbol{x}$ one-hot (single word)

Both learn the same factorized matrix $M = W_{\text{in}} W_{\text{out}}^T$ with $\text{rank}(M) \le d$; only the input vector that multiplies it changes.

3. Why Skip-gram tends to help rare words (a common intuition)

CBOW: a rare word is one of C inputs; its signal is mixed into an average dominated by frequent neighbours, and its row gets $\boldsymbol{g}/C$.

Skip-gram: each occurrence of a rare word makes it THE input for C separate targets, so its row receives every signal undiluted.

Main idea CBOW asks “which word fits this context?”; Skip-gram asks “which words surround this word?”.

Either way, the prediction task is a scaffold: the lasting product is the embedding table W_{in}.